

SENA — ADSO 2026
INFORME TÉCNICO

Investigación y Consumo de API Externa

Google Calendar API + Proyecto Texticode

Instructor: Ing. Edwin Marín | Mayo 2026

1. API SELECCIONADA: GOOGLE CALENDAR API

1.1 Justificación de la elección

Google Calendar API (Google Workspace) es la API de gestión de calendarios y eventos más utilizada a nivel empresarial. Para el proyecto Texticode — sistema de gestión de producción textil — su integración es completamente natural porque el sistema ya maneja fechas de entrega, turnos de operarios y órdenes de producción, pero carece de un mecanismo de planificación visual y recordatorios automáticos.

Necesidad en Texticode	Solución con Google Calendar API
Admin necesita planificar fechas de entrega de órdenes	Crear evento en calendario del admin con fecha límite de la orden
Operarios deben saber su turno de producción asignado	Recordatorio automático: popup 24 h antes y email 48 h antes del vencimiento
Seguimiento visual de carga de trabajo semanal/mensual	Listar eventos del período para mostrar dashboard de producción
Rescheduling de órdenes atrasadas	Actualizar fecha del evento vía PATCH al reprogramar la orden

El flujo sería: Admin crea orden con fecha de entrega → backend llama Google Calendar API → se crea un evento en el calendario del equipo → los operarios asignados reciben invitación → al reprogramar una orden, el evento se actualiza automáticamente.

1.2 Problema que resuelve la API

Texticode actualmente no tiene visualización de la agenda de producción ni recordatorios automáticos para operarios. Toda la planificación se hace manualmente en hojas de papel o mensajes de WhatsApp. Google Calendar API resuelve:

- Creación automática de eventos de entrega vinculados a cada orden de producción
- Invitaciones automáticas a los operarios asignados a cada tarea
- Vista unificada del calendario de producción para el equipo
- Actualizaciones en tiempo real cuando una fecha de entrega cambia
- Recordatorios automáticos sin intervención manual del administrador

2. INVESTIGACIÓN TÉCNICA DE LA API

2.1 Documentación oficial

URL de documentación: <https://developers.google.com/calendar/api/v3/reference>

Autenticación implementada con OAuth 2.0 por usuario (fetch directo a la Google REST API)

2.2 Métodos HTTP y endpoints clave

Método	Endpoint	Uso en Texticode	Respuesta clave
POST	/calendars/{id}/events	Crear evento de entrega al registrar una nueva orden	201 + objeto event con id
GET	/calendars/{id}/events	Listar eventos del mes para dashboard de producción	items[] con eventos
PATCH	/calendars/{id}/events/{eventid}	Actualizar fecha de entrega si la orden se reprograma	200 + evento actualizado
DELETE	/calendars/{id}/events/{eventid}	Eliminar evento al cancelar una orden de producción	204 No Content

2.3 Autenticación

Google Calendar API usa OAuth 2.0 como mecanismo de autenticación. La implementación real en Texticode usa el flujo OAuth 2.0 por usuario (Authorization Code Flow), donde cada usuario vincula su cuenta de Google individualmente. Los tokens se almacenan en la tabla `google_calendar_tokens` de la base de datos PostgreSQL:

- `access_token` y `refresh_token` almacenados cifrados en la BD — nunca en el frontend
- El flujo inicia en GET `/api/google/auth-url`, redirige a Google y recibe el código en GET `/api/google/callback`
- El `refresh_token` permite renovar el `access_token` automáticamente sin que el usuario vuelva a autorizar
- Scopes requeridos: `openid`, `email`, `profile`, `calendar` y `calendar.events` — solicitados en el prompt de consentimiento

```
// Autenticación OAuth 2.0 por usuario (Node.js - fetch directo a Google REST API)
// 1. Redirigir al usuario a Google para autorizar
// GET /api/google/auth-url → genera URL de consentimiento OAuth 2.0

// 2. Intercambiar código por tokens (en el callback)
const res = await fetch('https://oauth2.googleapis.com/token', {
  method: 'POST',
  body: new URLSearchParams({ code, client_id, client_secret,
    redirect_uri, grant_type: 'authorization_code' })
}); // Retorna access_token + refresh_token → se guardan en BD
```

2.4 Límites de uso y políticas

Aspecto	Detalle
Plan gratuito	1.000.000 de solicitudes por día. Completamente gratuito para proyectos de bajo volumen.
Rate limiting	500 solicitudes/100 segundos por proyecto. Prácticamente sin límite para una empresa PYME.
Cuota por usuario	250 solicitudes/100 segundos por usuario de Google. Para Texticode con 20-30 usuarios, no representa limitación real.
Tiempo de respuesta	Menos de 300 ms promedio para operaciones CRUD de eventos.
Almacenamiento	Sin límite por número de eventos. Google Drive gestiona el almacenamiento subyacente.

3. IMPLEMENTACIÓN TÉCNICA

3.1 Estructura del proyecto de demostración

```
texticode-backend/
├── .env                # GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI
├── .gitignore          # .env incluido - nunca subir credenciales al repositorio
├── README.md
├── index.js            # Servidor Express
├── db.js               # Conexión PostgreSQL (tabla google_calendar_tokens)
├── routes/google.js    # GET /auth-url, GET /callback, POST /sync/delivery-events
├── routes/
└── GET /events/upcoming, DELETE /unlink, GET /connected-users (solo admin)
```

3.2 Consumo funcional — Crear evento de entrega (POST)

Este endpoint POST crea un evento en Google Calendar cada vez que el admin registra una nueva orden con fecha de entrega en Texticode:

```
// routes/google.js - POST /api/google/sync/delivery-events
import express from 'express';
import db from '../db.js';
const router = express.Router();
// Lee el access_token del usuario desde la BD y lo usa
// directamente en el header Authorization: Bearer {token}

router.post('/crear-evento', async (req, res) => {
  const { ordenId, clienteNombre, productos, fechaEntrega, emailOperario } = req.body;

  const evento = {
    summary: `Texticode - Entrega Orden #${ordenId}: ${clienteNombre}`,
    description: `Productos: ${productos}\nCliente: ${clienteNombre}`,
    start: { dateTime: new Date(fechaEntrega).toISOString(), timeZone: 'America/Bogota' },
    end: { dateTime: new Date(new Date(fechaEntrega).getTime() + 3600000).toISOString(),
          timeZone: 'America/Bogota' },
    attendees: [{ email: emailOperario }],
  };
});
```

```
reminders: {
  useDefault: false,
  overrides: [
    { method: 'email', minutes: 2880 }, // 48 horas antes
    { method: 'popup', minutes: 1440 } // 24 horas antes
  ]
}
};

try {
  const response = await calendar.events.insert({
    calendarId: process.env.CALENDAR_ID,
    resource: evento,
    sendUpdates: 'all' // envía invitación al operario
  });
  res.json({ success: true, eventId: response.data.id });
} catch (error) {
  res.status(500).json({ success: false, error: error.message });
}
});
```

3.3 Consumo GET — Listar eventos del período

```
router.get('/eventos', async (req, res) => {
  const { desde, hasta } = req.query;

  try {
    const response = await calendar.events.list({
      calendarId: process.env.CALENDAR_ID,
      timeMin: new Date(desde).toISOString(),
      timeMax: new Date(hasta).toISOString(),
      singleEvents: true,
      orderBy: 'startTime'
    });

    const eventos = response.data.items.map(e => ({
      id: e.id,
      titulo: e.summary,
      inicio: e.start.dateTime,
      fin: e.end.dateTime,
      operario: e.attendees?.[0]?.email || null
    }));

    res.json({ total: eventos.length, eventos });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});
```

3.4 Actualizar fecha de entrega (PATCH)

```
router.patch('/actualizar-evento', async (req, res) => {
  const { eventId, nuevaFecha } = req.body;

  try {
    const response = await calendar.events.patch({
      calendarId: process.env.CALENDAR_ID,
```

```

    eventId,
    resource: {
      start: { dateTime: new Date(nuevaFecha).toISOString(), timeZone: 'America/Bogota'
    },
    end: { dateTime: new Date(new Date(nuevaFecha).getTime() +
3600000).toISOString(),
      timeZone: 'America/Bogota' }
    },
    sendUpdates: 'all' // notifica automáticamente al operario
  });
res.json({ success: true, evento: response.data });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

```

4. SEGURIDAD Y BUENAS PRÁCTICAS

4.1 Protección de credenciales

- Los access_token y refresh_token se almacenan en la tabla google_calendar_tokens de PostgreSQL — nunca en el frontend ni en el repositorio
- GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET se guardan en .env — nunca se suben al repositorio
- En producción se usan variables de entorno del servidor (Render, Railway, etc.)
- Cada usuario solo accede a su propio calendario (Calendar_Id = 'primary'), vinculado a su cuenta individual de Google

```

# .env — archivo local, NUNCA subir al repositorio
GOOGLE_CLIENT_ID=tu_client_id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=tu_client_secret

# .gitignore
.env
node_modules/
GOOGLE_REDIRECT_URI=http://localhost:3001/api/google/callback

```

4.2 Riesgos de seguridad identificados

Riesgo	Mitigación aplicada
Exposición de GOOGLE_CLIENT_SECRET en el repositorio	Variables de entorno en .env con .gitignore. Los tokens de usuario se almacenan en BD, no en el cliente.
Acceso no autorizado al calendario de producción	Cada usuario solo accede a su propio calendario. El admin puede ver usuarios vinculados desde el panel.
Spam de eventos o abuso del endpoint POST	Validar ordenId, clienteNombre y fechaEntrega antes de llamar la API. Autenticar el backend.

Cuota agotada por bucle de solicitudes	Implementar retry con exponential backoff ante errores 429 de la API.
--	---

5. PREGUNTAS DE INVESTIGACIÓN

P1. ¿Qué problema resuelve la API en Texticode?

Texticode gestiona órdenes y fechas de entrega, pero toda la planificación es manual — sin recordatorios ni vista de agenda. El admin y los operarios dependen de notas físicas o mensajes de WhatsApp para saber qué producir y cuándo. Google Calendar API automatiza la creación de eventos vinculados a cada orden, envía invitaciones a los operarios asignados y actualiza las fechas automáticamente al reprogramar. Esto elimina la planificación manual y evita olvidos de entregas.

P2. Métodos HTTP y datos que retorna

POST /calendars/{id}/events: recibe un objeto JSON con summary, description, start, end, attendees y reminders. Retorna HTTP 201 con el objeto completo del evento incluyendo su id único. GET /calendars/{id}/events: retorna un arreglo items[] con los eventos del período solicitado, incluyendo summary, start, end, attendees y status. PATCH /calendars/{id}/events/{eventId}: acepta campos parciales del evento y retorna el objeto actualizado con HTTP 200. DELETE retorna 204 sin cuerpo.

P3. ¿Cómo se autentica?

Mediante OAuth 2.0 Authorization Code Flow por usuario. El usuario hace clic en “Vincular con Google”, es redirigido a la pantalla de consentimiento de Google, y al autorizar se obtiene un access_token y un refresh_token. Ambos tokens se guardan en la tabla google_calendar_tokens de PostgreSQL asociados al usuario de Texticode. En cada sincronización, el backend lee el token del usuario, lo renueva automáticamente si expiró (usando el refresh_token), y lo usa directamente en el header Authorization de las llamadas fetch a la Google Calendar REST API. No se usa el SDK googleapis sino fetch nativo de Node.js.

P4. Límites gratuitos y políticas de uso

Google Calendar API ofrece 1.000.000 de solicitudes gratuitas por día. Para Texticode con 20-30 órdenes mensuales, el uso real sería de aproximadamente 100-200 llamadas al mes, siendo el plan gratuito completamente suficiente incluso a escala empresarial PYME. La cuota se gestiona desde Google Cloud Console y se puede solicitar aumento sin costo para proyectos justificados.

P5. Riesgos de seguridad

El principal riesgo es la exposición de GOOGLE_CLIENT_SECRET en el repositorio de código, lo que permitiría a terceros suplantar la aplicación y obtener tokens de usuarios. Un segundo riesgo es el robo de

los tokens almacenados en la base de datos: si la BD es comprometida, un atacante podría acceder a los calendarios de los usuarios. Ambos se mitigan con .env en .gitignore, tokens almacenados en BD sin exponerlos al frontend, y el refresh_token se usa únicamente en el backend para renovar el access_token de forma silenciosa.

P6. Mejora propuesta para Texticode

La integración ya fue implementada en Texticode con un flujo OAuth 2.0 por usuario. Cada usuario vincula su cuenta de Google desde el panel, y al ejecutar “Sincronizar ahora”, el backend recorre las órdenes vigentes del usuario, crea o actualiza los eventos en su Google Calendar personal (usando extendedProperties para evitar duplicados), y configura recordatorio por popup 24 horas antes y por email 48 horas antes. Los eventos se colorean automáticamente según prioridad: rojo para Alta, amarillo para Media y verde para Baja. Próxima mejora: sincronización automática al crear o modificar una orden, sin necesidad de acción manual del usuario.

6. EVIDENCIAS DE PRUEBA

6.1 Panel de integración Google Calendar en Texticode (Panel Administrador)

La siguiente captura muestra el módulo Google Calendar integrado en el Panel Administrador de Texticode. Se observa la cuenta conectada, la configuración de recordatorios (Email + ventanas emergentes) y la sincronización de fechas límite de las órdenes activas con opciones para añadir a Google Calendar o descargar en formato .ics.

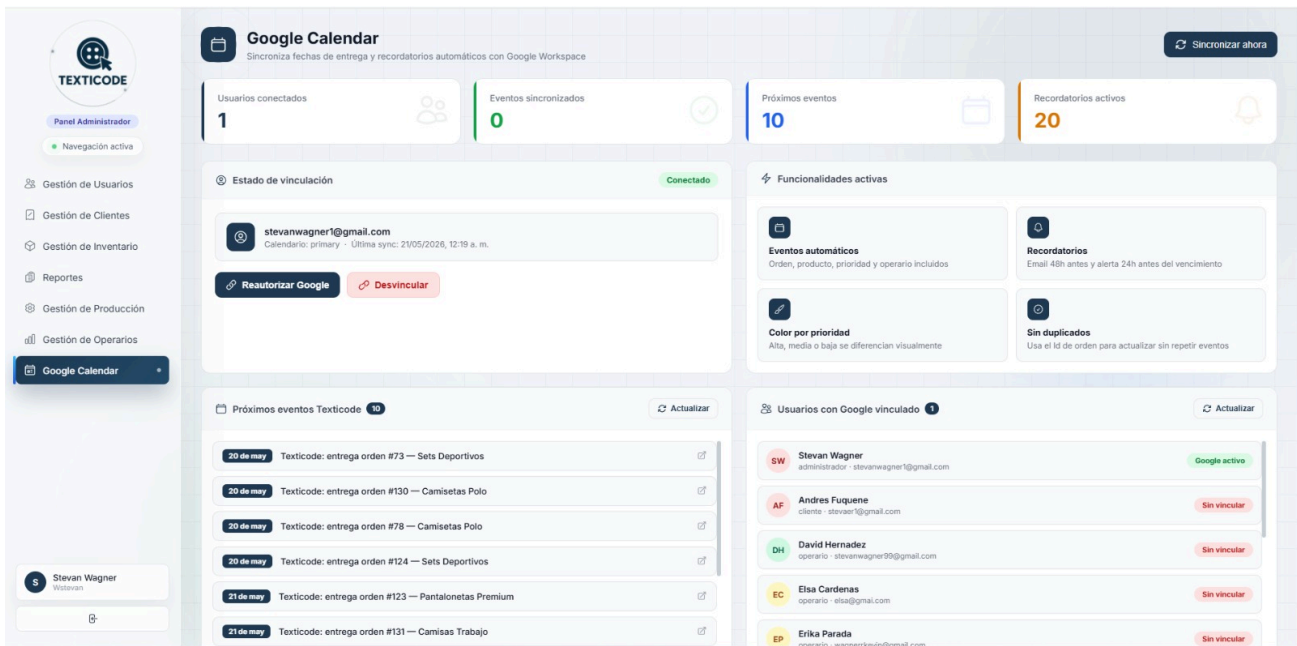


Figura 1 — Módulo Google Calendar en el Panel Operario de Texticode. Cuenta conectada, recordatorios configurados y sincronización de órdenes #10, #12 y #11.

6.2 Eventos sincronizados en Google Calendar —

La siguiente captura muestra el resultado directo de la integración: los eventos de producción de Texticode aparecen automáticamente en Google Calendar. Se puede ver la Orden #124 - Sets deportivos (21 de mayo), todos con la descripción completa que incluye cliente, producto, cantidad, prioridad y estado de la orden.

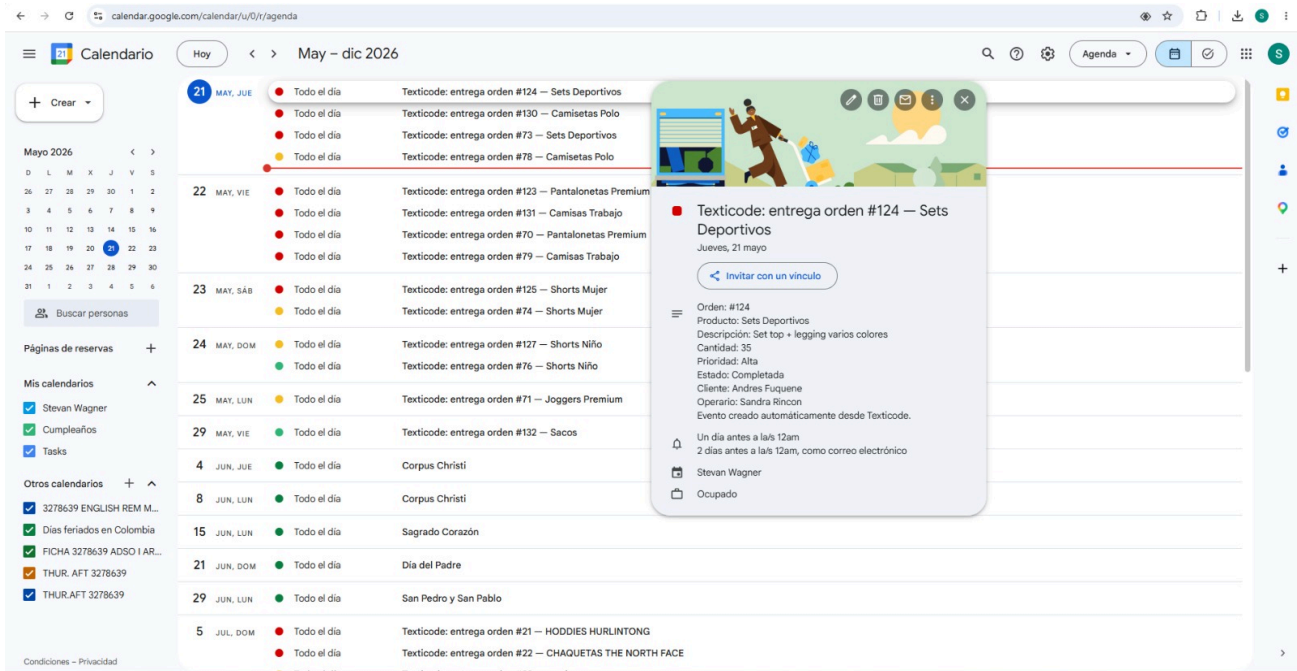


Figura 2 — Google Calendar de Abril 2026 con los eventos sincronizados desde Texticode. El detalle del evento muestra cliente, producto, cantidad, prioridad y recordatorios configurados (10 min, 1 hora y 1 día antes).

7. INTEGRACIÓN PROPUESTA EN TEXTICODE

7.1 Flujo de automatización completo

#	Actor	Acción
1	Admin	Crea orden con fecha de entrega en GestionProduccion.vue
2	Backend	POST /ordenes llama internamente a Google Calendar API para crear el evento
3	Google Calendar	Crea el evento y envía la invitación al email del operario asignado
4	Operario	Recibe la invitación en su Google Calendar y los recordatorios automáticos
5	Admin	Reprograma la fecha de una orden desde el panel de Texticode
6	Backend	PATCH /ordenes/:id actualiza la BD y llama PATCH a Google Calendar API
7	Operario	Recibe notificación automática de cambio de fecha en su calendario

Nota: sobre la elección de Google Calendar API vs otras APIs

Se descartó SendGrid porque ya fue documentada en el informe anterior. Se descartó OpenWeather (sin lógica de negocio en Texticode) y GitHub API (no aplica). Google Calendar API es la API con mayor coherencia funcional en este contexto: resuelve un problema real del sistema (planificación y recordatorios de órdenes), mejora el flujo existente (ciclo de vida de la orden con agenda visual), tiene un millón de solicitudes gratuitas al día y está completamente integrada con el ecosistema Google que ya usan las PYMES colombianas.